LLM-Based NER System for Medieval Historical Texts

Architecture, Runtime Modes, and Engineering Results

Rui Wang[1]

rwa041@uib.no

[1]**Digital development group, education and research support section, University of Bergen Library, Bergen, Norway**

15.04.2026

Agenda

1. Problem and Motivation
2. How the System Is Structured
3. How the System Processes Text
4. From Sequential to Concurrent Execution
5. Engineering and Results

Problem and Motivation

What Is Named Entity Recognition?

- Named Entity Recognition (NER) is a Natural Language Processing (NLP) task to identify and classify named entities in unstructured text data.
- It classifies these entities into categories such as person, place, organization, etc.
- For historical research, NER is a first step for information extraction across large text corpora — identifying who was mentioned, where, when, and in what role.

NER in This Project

- In this project, one CSV row is one historical record to process.
- The system reads the raw record text from the `Tekst` field.
- It then produces annotated text plus structured entity metadata.
- More than 18000 records need processing, so automation is essential.

Sample Input CSV

Header:
"Bindnr";"Brevid";"Tekst"

Example row:
"1";"601";"Ollum monnum þæim sæm þetta bref sea æder højra sændir Olauer med gudz nadh abote . . . "

The NER step must recognize entities inside this long historical text and return them in a reusable structured format.

Why This Project Is Difficult

- The source material is medieval historical text: spelling varies, text styles differ, and languages mix.
- Large historical corpora: manual annotation is infeasible, and error-prone for non-experts.
- Domain expertise is needed to understand context and annotate entities correctly.
- Existing NER tools might struggle with unstandardized and mixed-language input.
- LLMs are an option to handle complex language patterns at scale, but their outputs still require careful prompting, validation, and post-processing.

Operational Challenges

- Corpus file with more than 18000 historical text records.
- At scale, baseline per-record API processing and sequential execution become time-consuming.
- API-backed processing also has direct monetary cost, so prompt shape and execution mode matter.
- Concurrency and batch execution improve scalability, but they also add orchestration complexity.
- The system must balance throughput, cost, concurrency limits, failure handling, and output ordering.

Core Idea: Concurrency

It means a program can make progress on multiple tasks during the same period by switching when one task is waiting. This helps I/O-bound work such as network requests, but it is not the same as multi-core parallelism.

Key Terms in Python

- **Coroutine**: an `async def` function that can pause at `await`.
- **Task**: a scheduled coroutine managed by the event loop.
- **Event loop**: the `asyncio` runtime that switches between waiting tasks.

This project uses `asyncio` from the Python standard library to overlap LLM calls, batch polling, and fallback processing.

Batch Processing and Claude Message Batches API

Core Idea: Batch Processing

It submits many requests together for asynchronous processing. It is useful when immediate responses are not required and throughput or cost matters more than per-request latency.

Message Batches API

- The system creates a Message Batch with a list of requests.
- Each request is processed asynchronously and handled independently by Claude.
- The status of the batch can be polled and results can be retrieved when processing has ended for all requests.
- Results are not guaranteed to come back in input order, so `custom_id` is used to match them back.

Python SDK Structure

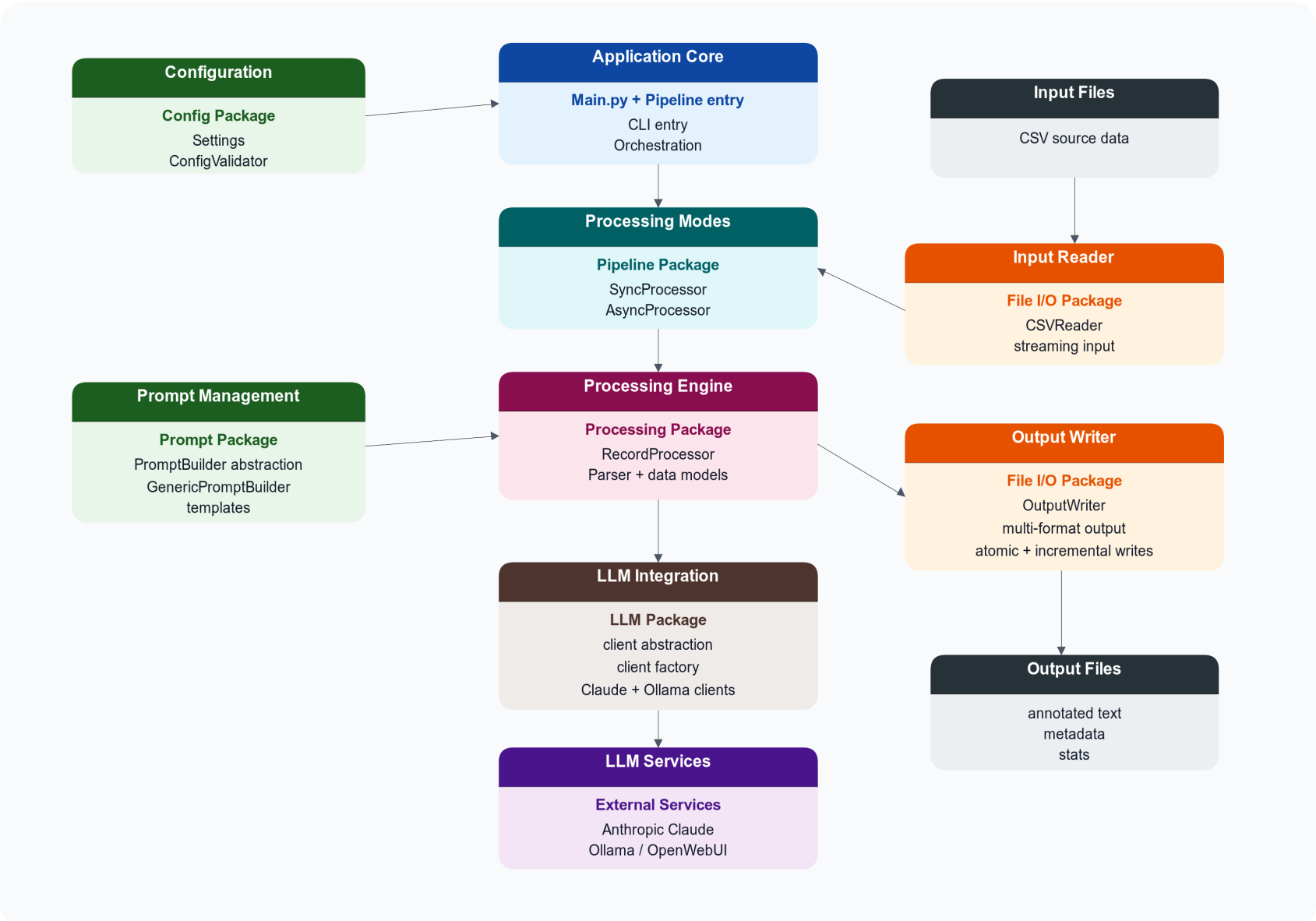
```
batch = client.messages.batches.create(
  requests=[
    {
      "custom_id":  "record-601",
      "params":  { model, max_tokens,
         messages:  [{...}] }
    },
    {
      "custom_id":  "record-602",
      "params":  { ...  }
    }
  ]
)
```

Runtime Pattern

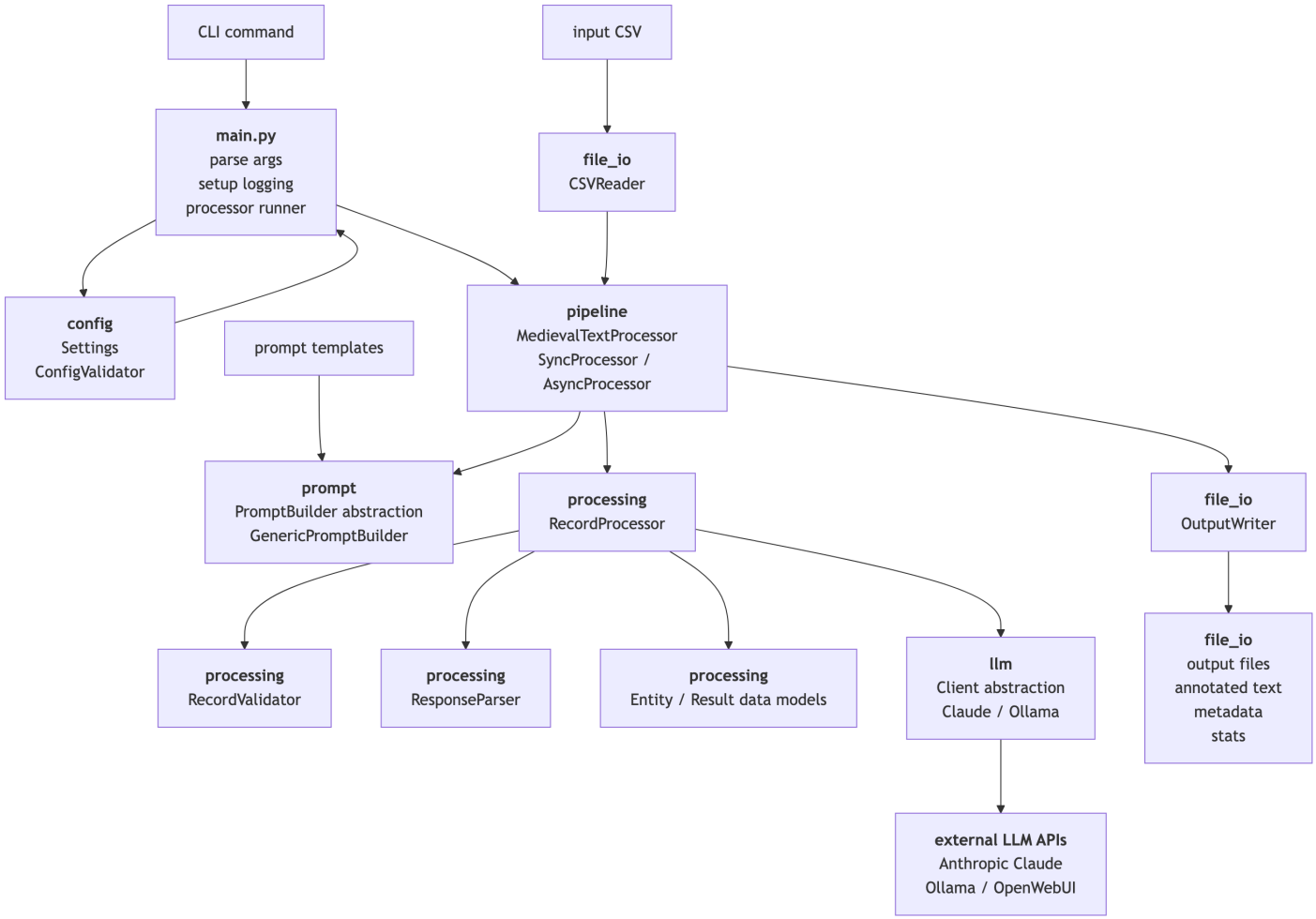
`create batch → poll status → fetch results`

How the System Is Structured

System Architecture Overview

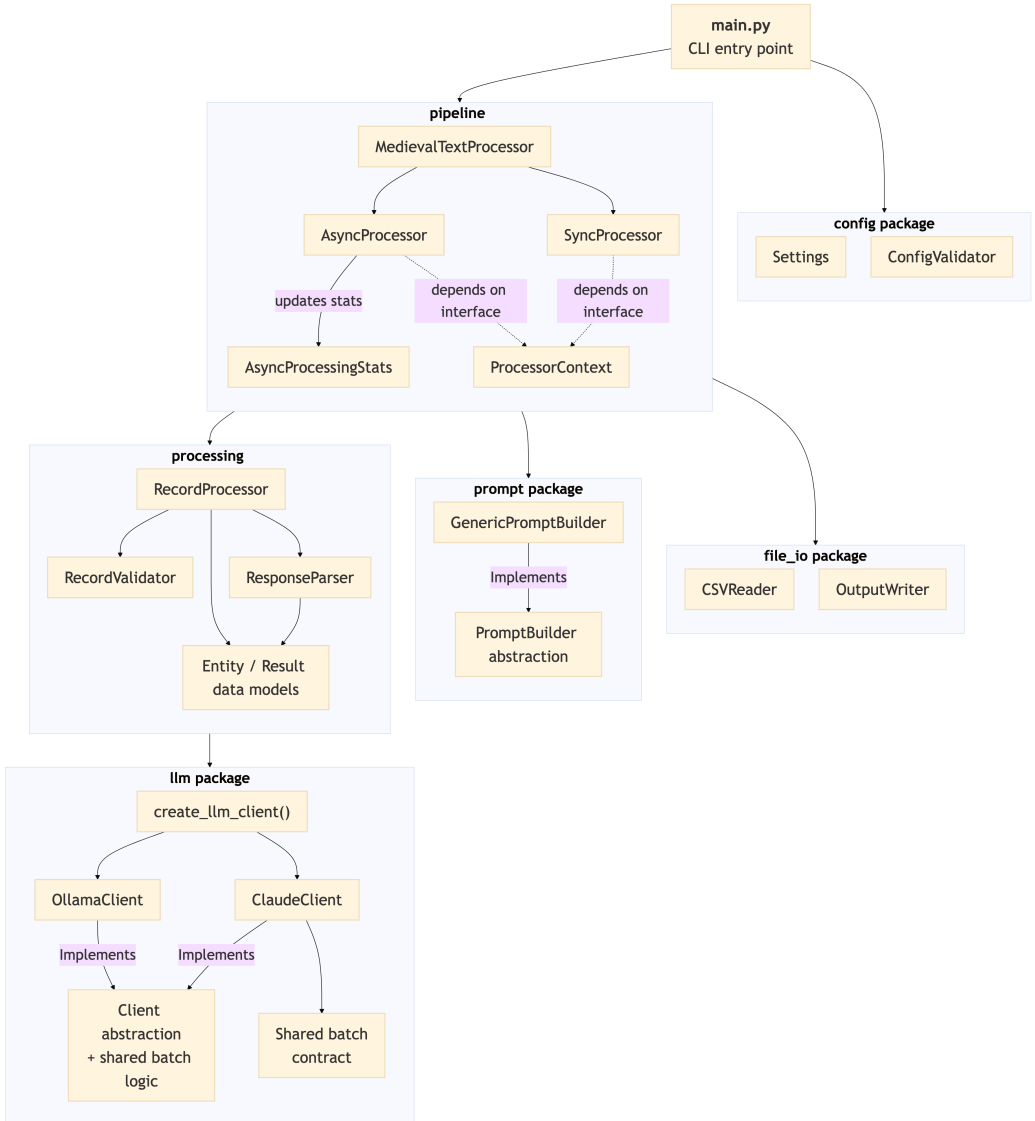


Runtime Execution Flow



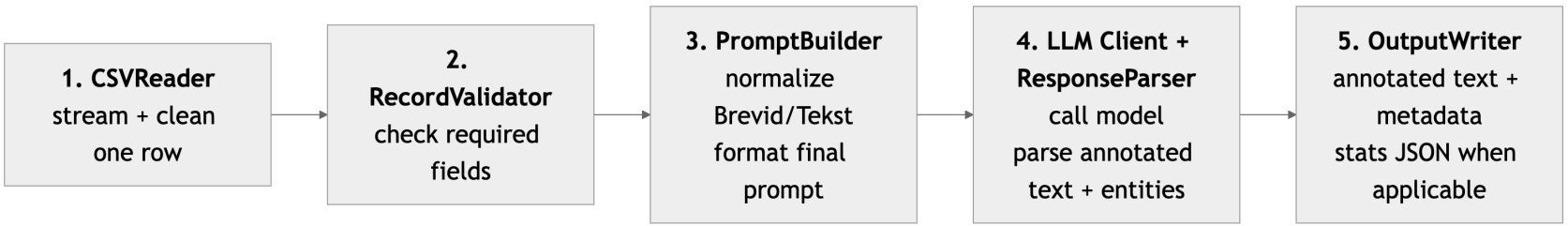
The application flows from CLI and configuration into pipeline orchestration, record processing, LLM calls, and final output files.

Package-Oriented Architecture



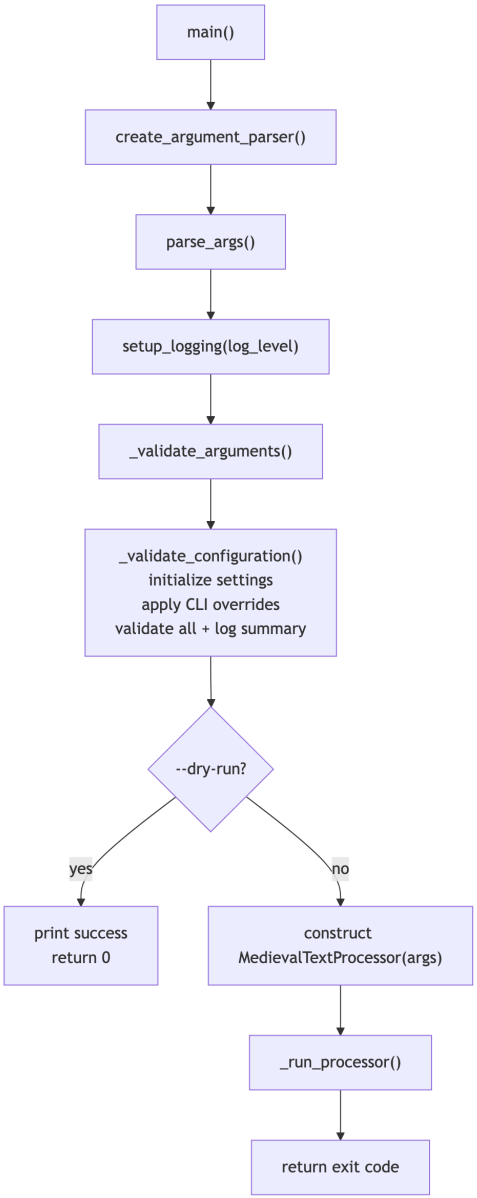
`main.py` starts the app; the six packages divide configuration, I/O, prompts, LLM integration, processing logic, and orchestration responsibilities.

End-to-End Record Journey



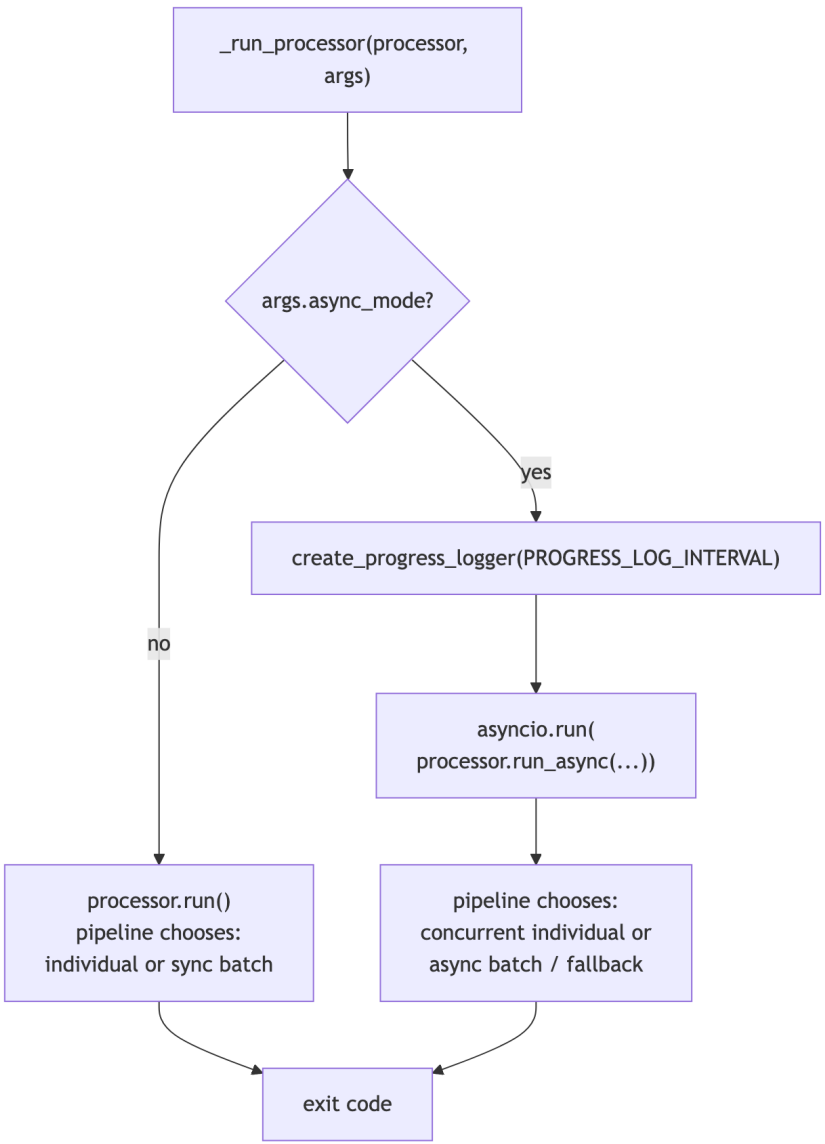
One cleaned CSV row follows the same validate-build-call-parse path in both sync and async modes; the pipeline only changes orchestration and write strategy.

`main.py` Role and Startup Flow



- `main.py` is the CLI entry point and top-level exit-code boundary.
- It builds the parser, parses arguments, configures logging, and performs quick argument checks.
- `_validate_configuration()` initializes `Settings`, applies CLI overrides, runs `ConfigValidator.validate_all()`, and logs the effective configuration.
- `-dry-run` exits after full validation, before processor construction or any model call.
- On success, `main()` constructs `MedievalTextProcessor` and delegates execution.

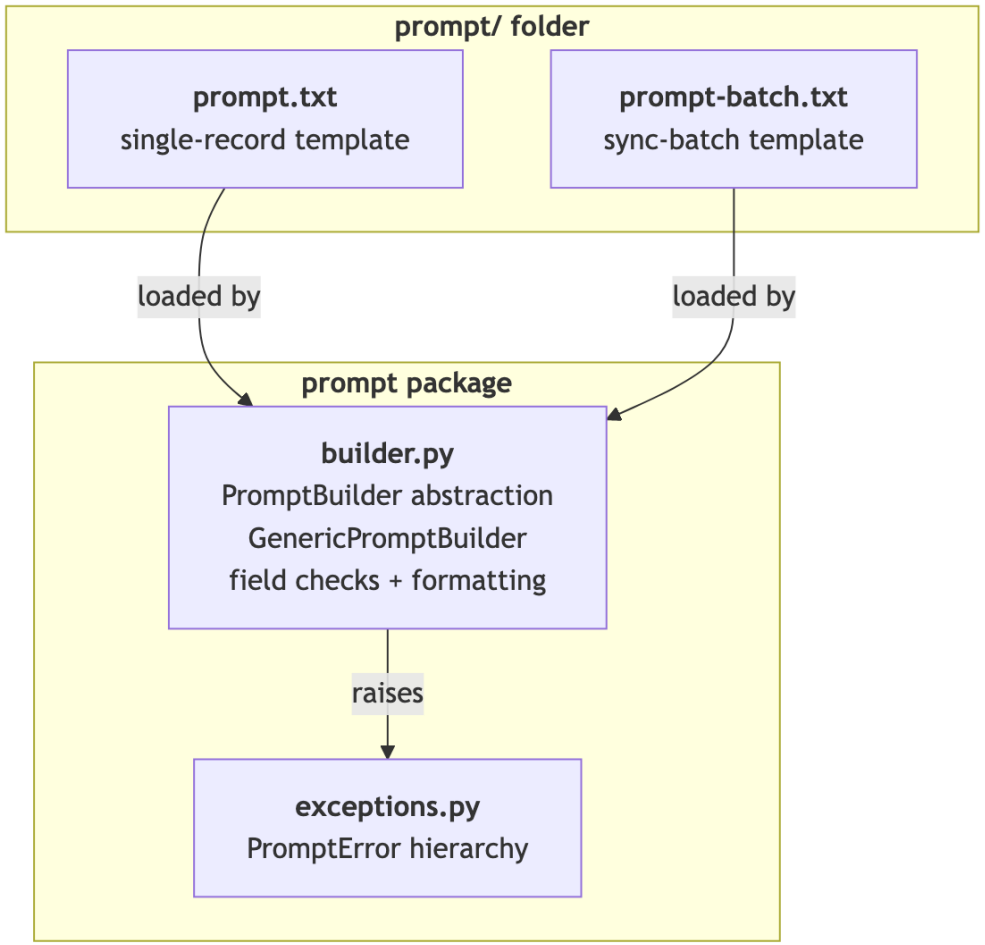
Sync vs Async Dispatch and Processing Modes



- `main.py` performs only the top-level sync vs async dispatch.
- Sync mode calls `processor.run()`, where the pipeline chooses individual processing or sync batch processing.
- Async mode creates a progress callback, then uses `asyncio.run(...)` to execute the `processor.run_async(...)` coroutine.
- Inside async mode, the pipeline chooses concurrent individual processing or async batch orchestration, depending on client capability and options.
- So `main.py` owns CLI dispatch and event-loop entry; the pipeline owns the real execution strategy.

How the System Processes Text

Prompt Package Overview



- The top-level `prompt/` folder stores the actual template files.
- The prompt package loads those files, extracts placeholders, and formats final prompts.
- The caller chooses the template file; `build(data)` dispatches by input shape inside `GenericPromptBuilder`.
- It normalizes source keys `Brevid/Tekst` to template keys `brevid/text` and checks required fields before formatting.
- `exceptions.py` defines prompt-specific template-load and prompt-build errors.

Single-Record Template: `prompt.txt`

Core Structure from `prompt.txt`

- `Brevid: {brevid}`
- `Text: """{text}"""`
- Task 1 marks up all proper nouns in the full medieval text as the format expression: < name;type;preposition;order;brevid >.
- Task 2 returns `===JSON===` with entity metadata: `name`, `type`, `preposition`, `order`, `description`, `brevid`, `gender`, and `language`.

- The single-record template uses lowercase placeholders `{brevid}` and `{text}`.
- The output combines full annotated text and structured `===JSON===` metadata.
- It is used for sync single-record processing, async single-record processing, and async batch per-record requests.

Sync Batch Template: `prompt-batch.txt`

Core Structure from `prompt-batch.txt`

- Starts with strict output-format enforcement rules.
- Uses `{num_records}` to declare the batch size.
- Uses `{batch_content}` to inject all record payloads.
- Requires repeated `RECORD N RESULT` sections with annotated text plus `===JSON===` entity output.

- The synchronous batch template uses `{num_records}` and `{batch_content}` placeholders.
- The builder assembles `batch_content` as repeated `RECORD i`, `Brevid`, and `Text` input blocks.
- The response is structured as repeated `RECORD N RESULT` sections.
- It is only for synchronous multi-record prompting; Claude async batch still sends one prompt per record.

Prompt Flow in Code

Template Path

```
prompt/prompt.txt
prompt/prompt-batch.txt
```

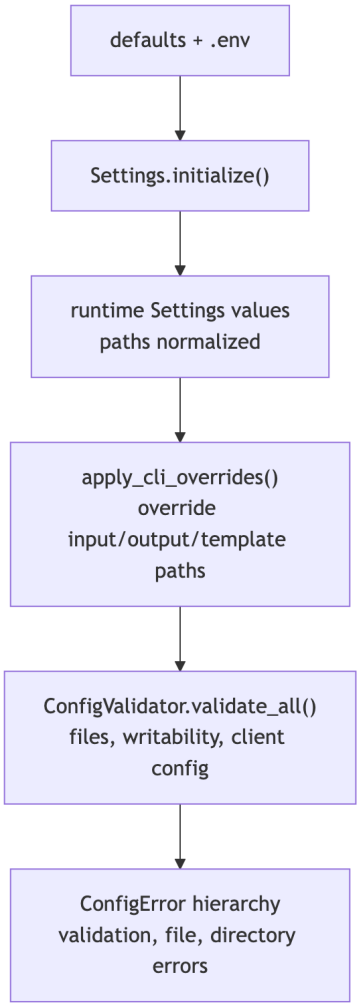
Builder Logic Pseudocode

```
procedure INIT(template_file)
    load template

procedure BUILD(data)
    if single record then
        extract template placeholders
        require {brevid, text} placeholders
        validate and clean record
        format and return final prompt
    else
        require non-empty record list
        extract template placeholders
        require {num_records, batch_content}
placeholders
        for all records in batch do
            validate and clean record
        assemble batch content
        format and return final prompt
```

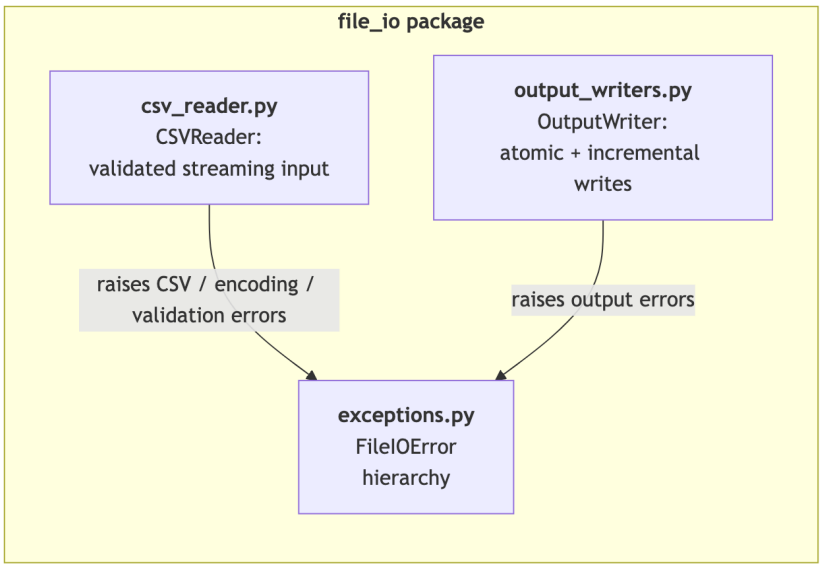
- The template file is chosen by the caller; `build(data)` dispatches by input shape.
- `build(data)` accepts either one record or a list of records.
- Single-record mode validates and formats `{brevid}` and `{text}`.
- Sync-batch mode validates `{num_records}` and `{batch_content}` and assembles repeated record blocks.
- Template loading fails early on missing, empty, non-file, or unreadable template files.
- Async batch still uses one single-record prompt per record.

Configuration Package



- `Settings` loads defaults and environment values into shared runtime configuration.
- `Settings.initialize()` can normalize paths, create output directories, and optionally validate common config.
- CLI overrides are applied afterward through `apply_cli_overrides()`, especially for input, output, and template paths.
- `ConfigValidator` is the pre-flight validation layer for client config, template/input files, and output path writability.
- `exceptions.py` defines the config error hierarchy for missing config, file problems, and directory problems.

File I/O Package



- `CSVReader` validates the input file, then streams cleaned semicolon-delimited rows.
- Required CSV headers can be validated when configured.
- `OutputWriter` writes three artifacts: annotated text, metadata table, and JSON stats.
- Text and metadata support atomic full writes and incremental writes with POSIX locked appends; stats use one final atomic JSON write.
- This package is the filesystem boundary of the application.

File I/O Runtime Logic

CSVReader Logic Pseudocode

procedure STREAM_RECORDS
    file already validated at construction
    open CSV file with configured delimiter and encoding
    create `csv.DictReader`
    capture headers
    validate required headers if configured
    for all rows in CSV **do**
        if row is empty **then**
            skip row
        else
            clean row values
            yield cleaned record
    wrap encoding, CSV, and OS errors as file I/O exceptions

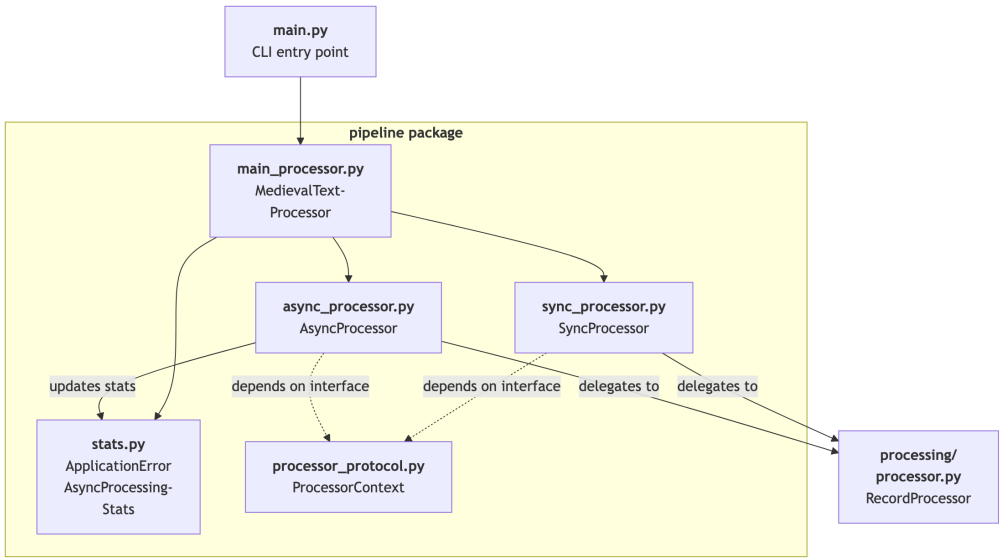
`OutputWriter` Write Strategy

- Full text/metadata writes: build file content and atomically replace the target file.
- Incremental text/metadata writes: lock file, add header only if empty at lock time, add one separator newline if needed, append chunk, flush, unlock.
- Stats: serialize JSON and write one final atomic file.
- Locked append mode depends on POSIX `fcntl`, so it is not the cross-platform path.

Why This Design

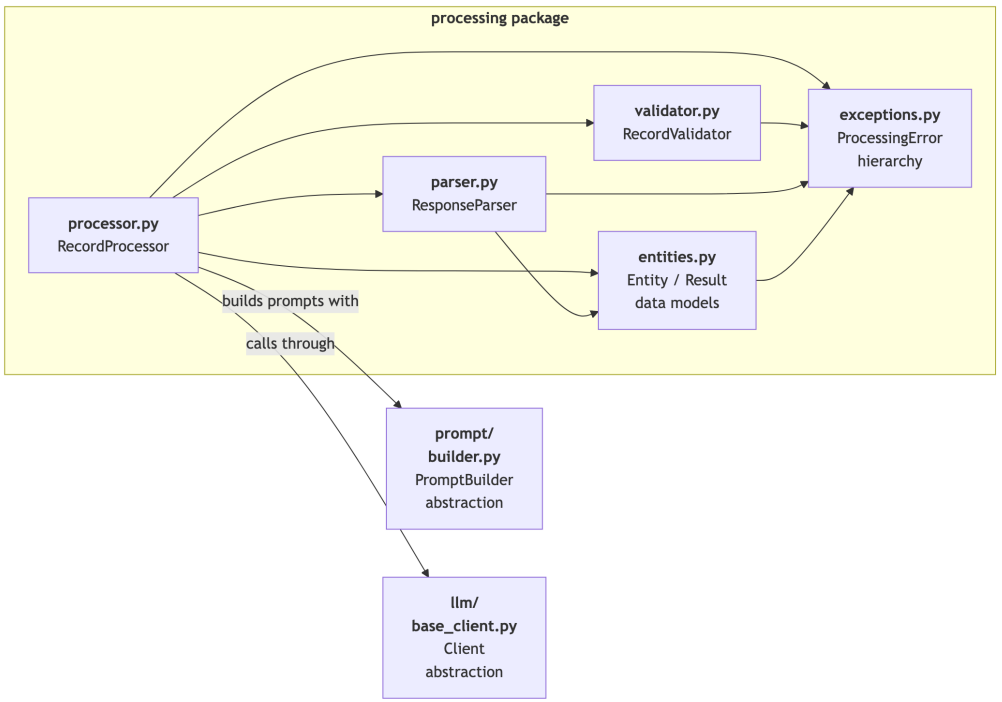
- Streaming keeps input memory usage bounded.
- Atomic full writes avoid partially replaced output files.
- Locked appends let async processing emit ordered records safely during long runs.

Pipeline Package



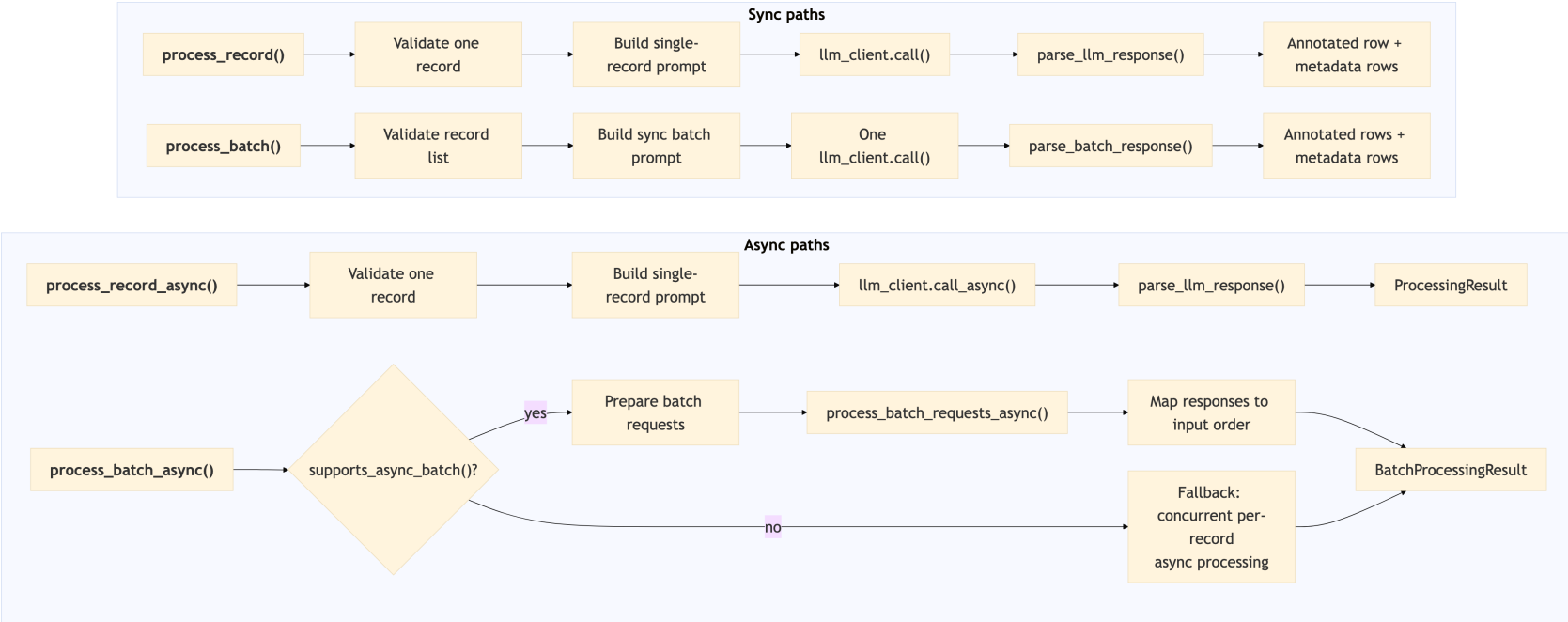
- `MedievalTextProcessor` is the top-level runtime coordinator: it wires shared components, owns sync/async entry points, and handles output cleanup/finalization.
- `SyncProcessor` runs streaming synchronous execution, including individual mode, sync batch mode, and fallback handling.
- `AsyncProcessor` runs concurrent async execution with order preservation, incremental output, and batch progress monitoring.
- `processor_protocol.py` defines `ProcessorContext`, a protocol-based interface so sub-processors depend on a narrow contract instead of the concrete main processor.
- `stats.py` holds async run statistics and the application-level error type.

Processing Package



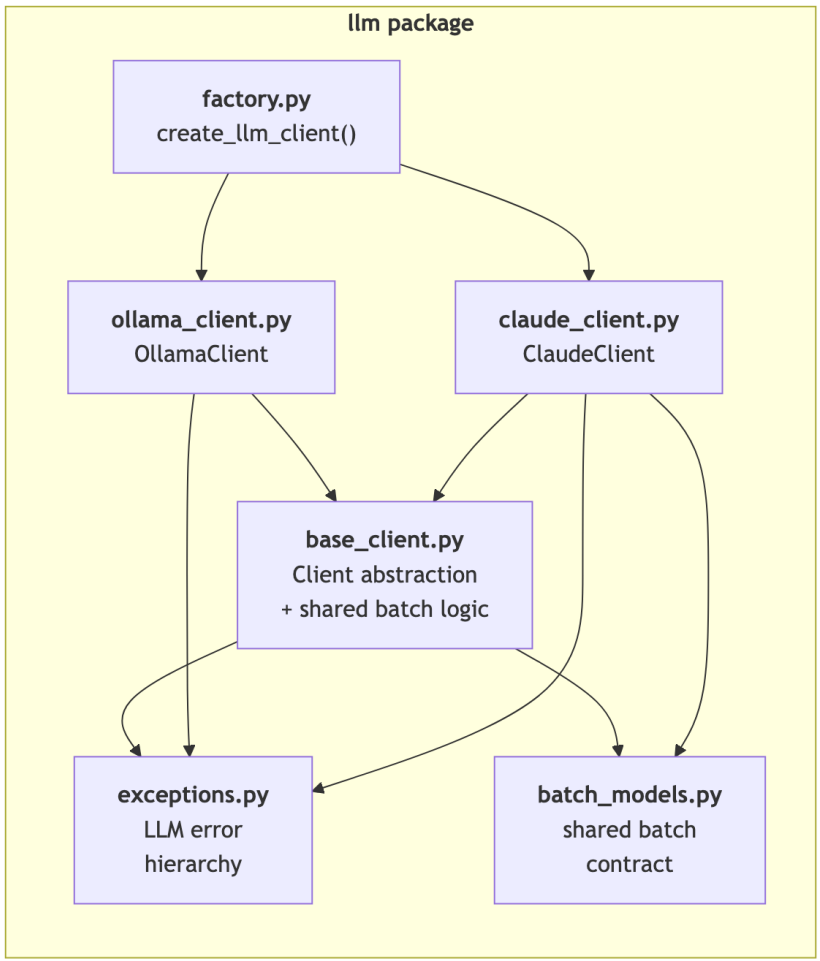
- This is the business-logic engine of the system.
- `RecordProcessor` orchestrates sync/async single-record and batch processing.
- `validator.py` checks input records, and `parser.py` converts raw LLM output into annotated text and entity metadata.
- `entities.py` defines `EntityRecord`, `ProcessingResult`, and `BatchProcessingResult`.
- `exceptions.py` centralizes the processing-layer error hierarchy.

RecordProcessor Flow



Sync and async entry points share the same core stages: validate input, build prompts, call the LLM, parse responses, and package typed outputs.

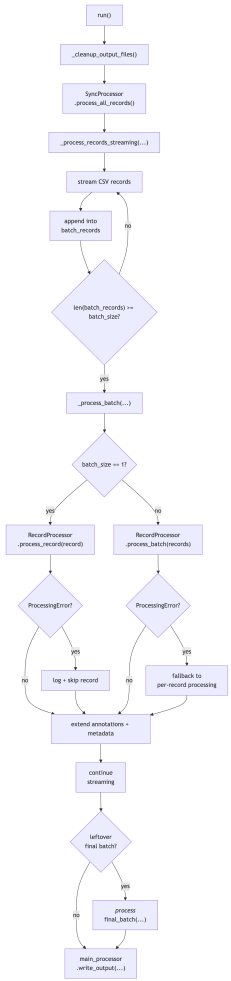
LLM Package



- The LLM package hides provider-specific APIs behind the shared `Client` abstraction.
- `base_client.py` defines the common contract and shared async batch orchestration workflow.
- `ClaudeClient` is the full-featured adapter: sync, async, and async batch support.
- `OllamaClient` supports sync and async single requests, but not async batch.
- `batch_models.py` and `exceptions.py` provide the shared batch contract and provider-level error hierarchy.
- `factory.py` creates the concrete client from validated settings.

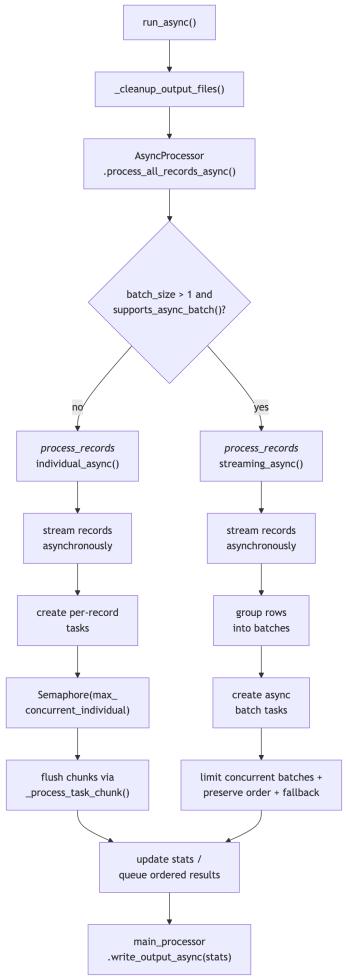
From Sequential to Concurrent Execution

Synchronous Pipeline Flow



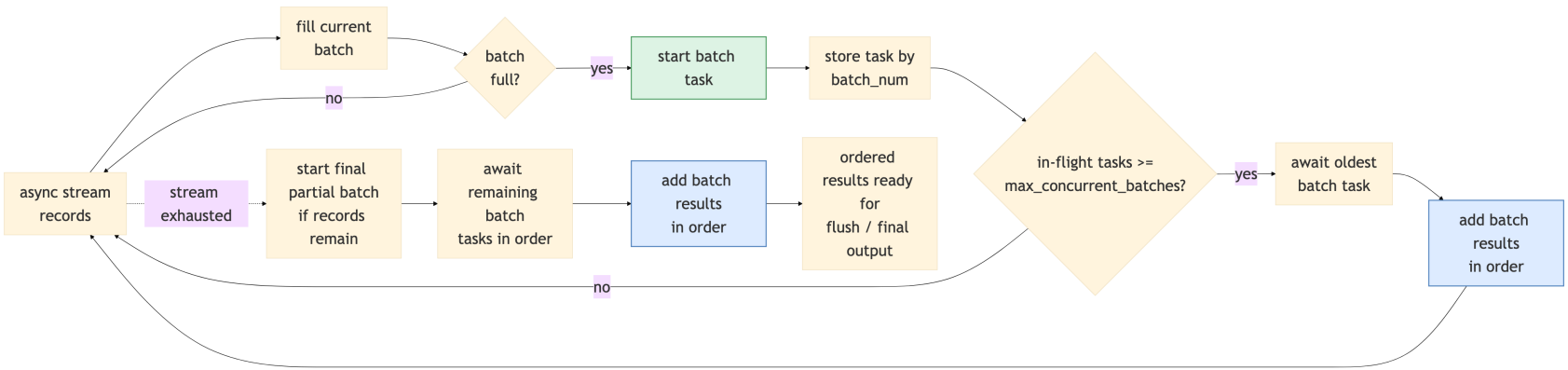
`main_processor.run()` cleans old outputs, then `SyncProcessor` streams records, falls back from failed batches to per-record processing, and writes output once at the end.

Asynchronous Pipeline Flow



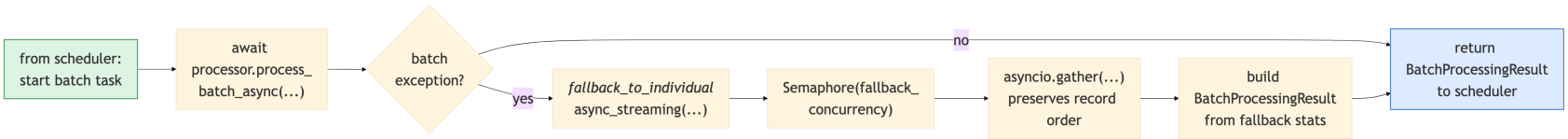
`main_processor.run_async()` cleans old outputs, then `AsyncProcessor` chooses between concurrent per-record processing and provider-supported async batch streaming before final output is written.

Async Batch Scheduler



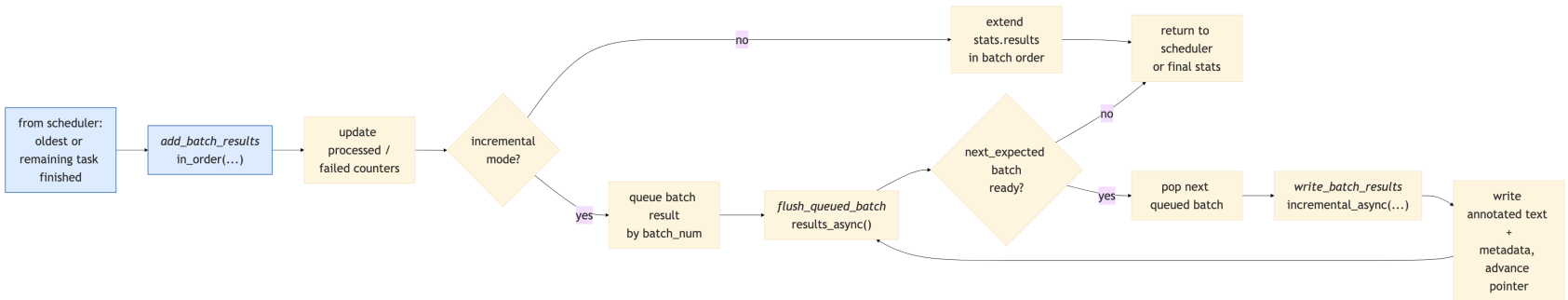
Step 1: fill batches, start tasks, and wait on the oldest task once the in-flight limit is reached. The next two slides zoom into the task body and ordered result handling.

Async Batch Task Fallback



Step 2: each started task either returns a normal `BatchProcessingResult` or falls back to per-record async processing. Either way, the scheduler receives the same result shape: a completed batch result returned for ordered handling on the next slide.

Async Batch Result Ordering



- Step 3 starts when the scheduler receives a completed batch result and calls `_add_batch_results_in_order()`.
- That method always updates counters before extending results or queueing the batch.
- In incremental mode, `_flush_queued_batch_results_async()` may write multiple consecutive ready batches while `_next_expected_batch_num` advances.

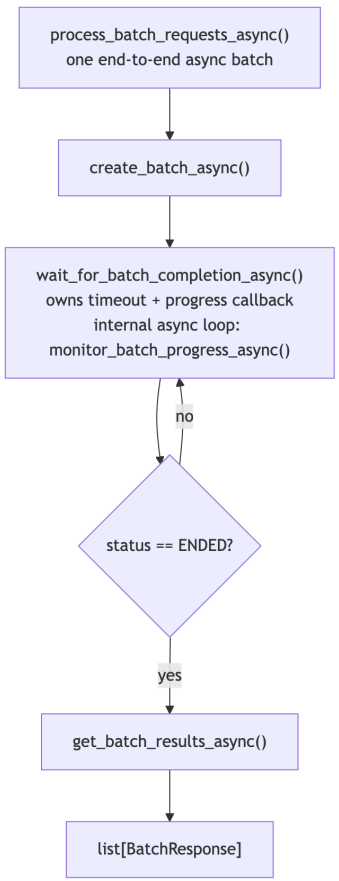
Async Batch Ordering Pseudocode

Core Scheduling Logic

```
procedure PROCESS_RECORDS_STREAMING_ASYNC
    for all streamed records do
        add record to current batch
        if batch full then
            start batch task and store by batch_num
            if in-flight limit reached then
                await oldest batch task
                add results in order
    start final partial batch if needed
    for all remaining batch tasks in order do
        await task
        add results in order
    if incremental mode then
        flush queued next-expected batches
```

- This condenses Step 1 and Step 3 into one control-flow view.
- Oldest-batch waiting preserves batch order while still allowing bounded concurrency.
- Failed batch tasks still return `BatchProcessingResult` after fallback, so the scheduler path stays uniform.
- Ordered flushing is separated from scheduling: ready batches are written only when their turn arrives.

Why Claude Async Batch Matters



- The shared async-batch orchestration is `create -> wait & poll -> fetch results` for one batch job.
- Claude lets the pipeline submit multiple batches concurrently instead of waiting on one combined prompt at a time.
- Ollama does not implement that API shape, so higher layers must fall back to non-batch async strategies.
- This provider difference is one reason the LLM client abstraction matters.

Engineering and Results

Project Setup and Run Workflow

- Install dependencies with `uv sync`.
- Copy `.env.example` to `.env` and add Claude API or OpenWebUI/Ollama credentials.
- Default client, paths, and templates come from settings and environment values; CLI flags can override them for custom runs.
- `-dry-run` performs full argument, config, file, and client validation without processing.
- Async mode also exposes tuning flags for concurrency, chunking, polling, wait limits, and incremental output.

Common Run Profiles

- **Sync individual**: default path
- **Sync batch**: `-use-batch` + `-batch-size N`
- **Async individual**: `-async-mode`
- **Async batch**: `-async-mode` + `-batch-size N`

Async batch still depends on provider capability: Claude supports it; Ollama falls back to non-batch async execution.

Example Commands

```
# Validate without processing
uv run -m ai_ner_system.main
  -client claude -dry-run
# Sync batch processing
uv run -m ai_ner_system.main
  -client ollama -use-batch
  -batch-size 10
# Async batch (production)
uv run -m ai_ner_system.main
  -client claude -async-mode
  -batch-size 100
  -incremental-output
```

Project Tooling

Runtime and Packaging

- Python requirement is `>=3.11`; CI also runs on 3.12 and 3.13.
- `uv` is the primary workflow for environment setup, dependency sync, and command execution.
- `pyproject.toml` is the single source of truth for package metadata, dependencies, entry points, and developer tooling.
- `uv.lock` pins resolved versions for reproducible environments and safer dependency updates.
- The CLI can run as `uv run ai-ner-system ...` instead of the module form.

Dependency and Workflow Groups

- Core runtime dependencies include `python-dotenv`, `anthropic`, `aiohttp`, and `requests`; `asyncio` comes from the Python standard library.
- Project extras expose install profiles such as `lint`, `test`, `security`, and combined `ci`.
- `uv` also defines development groups for the main engineering workflows used for this project.
- Static analysis and pytest behavior are configured centrally in `pyproject.toml`; the next slide shows how those checks run in practice.

Testing and Quality Assurance

Quality Toolchain

- **Ruff** — lint + format checks
- **MyPy + Pyright** — static typing
- **Pytest + pytest-asyncio** — unit and async tests
- **Pre-commit** — local gate before CI

CI and Repo Safeguards

- Push/PR CI runs Ruff, MyPy, and unit tests with coverage on Python 3.11 / 3.12 / 3.13, then uploads coverage to Codecov.
- Separate workflows run Bandit, Safety, and CodeQL.
- GitHub safeguards include Dependabot alerts, code scanning alerts, and code quality findings.

Coverage Snapshot by Package

`config`	95–100%
`file_io`	94–100%
`llm`	95–100%
`processing`	100%
`prompt`	90–99%
`pipeline`	0%
`main.py`	0%

- Latest recorded unit-test coverage is about 69% overall; five core packages are at 90–100%.
- `pipeline` and `main.py` are still the weakest areas because orchestration logic needs more direct tests.

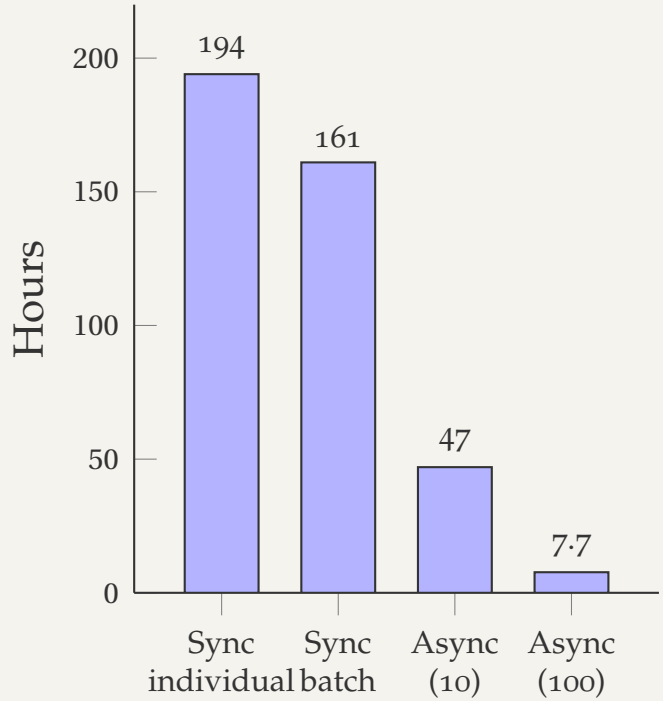
Performance Benchmarks

Benchmark Results (~18 559 record corpus)

Mode	Provider	Records	Time	Throughput	Cost	Projected
Sync individual	**Ollama/Gemma3**	10	6:15	1.6 rec/min	$0.31	~194 h
Sync batch (size 10)	**Ollama/Gemma3**	10	5:23	1.9 rec/min	$0.29	~161 h
Async batch (size 10)	**Claude/sonnet4**	10	1:34	6.4 rec/min	$0.17	~47 h
Async batch (size 100)	**Claude/sonnet4**	551	13:47	40 rec/min	~$12	~7.7 h

- These benchmarks compare both execution mode and provider/model, not batching strategy alone.
- The 551-record async row is a scale trial; the other rows are small-sample runs.
- Claude async batch shows the largest measured throughput gain, time reduction, and lower cost per record at scale.
- Architecture choice here affects runtime, cost, and feasible corpus-scale execution.

Projected Full-Corpus Time



Anthropic API Rate Limits

Claude API Limits (Tier 2)

Message Batches API	
Requests per batch (max)	100K
Requests in processing queue	200K
Management endpoint calls/min	1K
Messages API (per model)	
Requests per minute	1K
Input tokens per minute	450K
Output tokens per minute	90K
Spend	
Monthly spend limit	$500

Batch vs Management Calls

A batch contains up to 100K requests, but creating, polling, and retrieving that batch are **management calls** (1K/min limit). The batch contents are processed asynchronously by Anthropic's servers.

These Tier 2 limits are taken from the current Anthropic docs/console and may vary by organization or tier.

Concurrency Design for Staying Within API Limits

Batch Async Path (Batches API)

- **Bounded batch concurrency**: `max_concurrent_batches` (default 5) caps in-flight batch jobs. The scheduler awaits the oldest batch before starting a new one.
- **Poll-based progress monitoring**: `poll_interval` (default 30 s) controls how often each batch checks its status via the management API.
- **Fallback concurrency**: `fallback_concurrency` (default 3) limits parallelism when a failed batch retries records individually.

Individual Async Path (Messages API)

- **Semaphore-limited concurrency**: `max_concurrent_individual` (default 5) bounds concurrent per-record API calls.
- **Chunk-based memory management**: `chunk_size` (default 50) limits how many tasks accumulate before awaiting results together.

Management API Call Budget

```
Each poll = 2 API calls (status check + batch
info).  At poll_interval=30s, each batch polls
2 polls/min.  With 5 concurrent batches:
    5 batches × 2 calls/poll × 2 polls/min =
                20 calls/min
        (2% of 1K management limit)
```

All defaults are in `Settings` and overridable via CLI flags.

Candidate Optimized Settings for Full-Corpus Execution

These are modeled candidate settings from the current benchmarks and API limits, not yet validated by a full-corpus run.

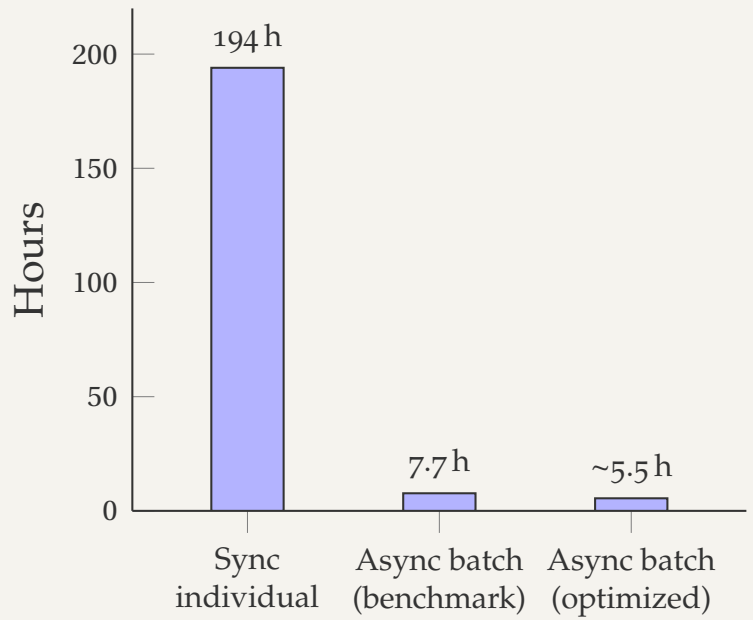
Default vs Optimized Settings

	Default	Optimized
`batch_size`	5	**200**
`max_conc._b.`	5	**8**
`poll_interval`	30 s	**20 s**
`chunk_size`	50	**100**
`fallback_conc.`	3	3

Why These Values Are Safe

- `batch_size=200`: 0.2% of the 100K/batch limit. Creates about 93 batches for 18,559 records.
- `max_concurrent_batches=8`: at most 1,600 records in queue (0.8% of 200K limit).
- Management API usage: ~56 calls/min (5.6% of 1K limit).
- Estimated cost: ~$396 (within the $500/month Tier 2 cap).

Projected Full-Corpus Time



- Benchmark (batch 100, 5 concurrent): 40 rec/min.
- Optimized (batch 200, 8 concurrent, 20 s poll): projected ~55 rec/min → **~5–6 hours**.

Annotated Output Example

Sample annotated text from a Claude async incremental run (13 records, batch_size=2).

Annotated Text Excerpt (Brevid 601)

```
...  sændir < Olauer;Person Name;N/A;1;601 >
med gudz nadh abote j < Olafsklaustre;Place Name;j;2;601 >
j < Tunsbergi;Place Name;j;3;601 > q.  g.  ok sina kunnikt gerande ...
...  opno brefue a < Olafsklausters;Place Name;a;4;601 > vægna þet ...
...  sæm sira < Æirikar Kolbiornason;Person Name;N/A;5;601 >
prester a < Sondum;Place Name;a;6;601 > hafde kœypt ...
```

Inline tags preserve the original text and add `< name; type; preposition; order; brevid >`.

Metadata Output Example

Sample metadata rows from the same Claude async incremental run.

Selected Metadata Rows

```
Entity;Type;Preposition;Order;Brevid;Description;Gender;Language
Olauer;Person Name;N/A;1;601;Abbot;Male;non
Olafsklaustre;Place Name;j;2;601;Monastery;N/A;non
Tunsbergi;Place Name;j;3;601;Town/City;N/A;non
Olafsklausters;Place Name;a;4;601;Monastery;N/A;non
Æirikar Kolbiornason;Person Name;N/A;5;601;Priest;Male;non
Sondum;Place Name;a;6;601;Location;N/A;non
```

One semicolon-delimited row is written for each extracted entity.

Future Work

Short-term

1. Add direct tests for pipeline orchestration and `main.py` paths.
2. Validate candidate optimized async-batch settings on the full 18,559-record corpus.
3. Evaluate output quality through domain review and structured error analysis.
4. Measure production-scale cost, throughput, and fallback behavior.

Long-term

1. Add smarter retry/backoff and adaptive rate-limit handling for provider APIs.
2. Expand integration/system testing for full sync/async workflows.
3. Explore **harness engineering**[1] to build an agentic architecture for medieval-text processing with reusable orchestration and experiment workflows.
4. Integrate the pipeline with a frontend or dashboard for easier monitoring and result inspection.

[1] Harness engineering: reusable orchestration, tool integration, and context management for agentic systems.